

Exploratory Data Analysis

Objective:

To explore the dataset for key trends, outliers, distributions, and data quality issues before building the Power BI dashboard and summarizing insights.

Data Quality Overview

Sample Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Loading private GCS files
customers = pd.read_csv("gs://ppnj_data/files/dim_customer.csv")
travel_package = pd.read_csv("gs://ppnj_data/files/dim_travel_package.csv")
sales = pd.read_csv("gs://ppnj_data/files/fact_sales.csv")
travel_guide = pd.read_csv("gs://ppnj_data/files/dim_travel_guide.csv")
feedback = pd.read_csv("gs://ppnj_data/files/dim_feedback.csv")
refund = pd.read_csv("gs://ppnj_data/files/dim_refund.csv")
```

Results:

Dataset	Rows	Key Notes
dim_customers	1234	No missing IDs or names, but email has 43 missing values (~3.5%). Consider cleaning/handling these for campaigns or contact flows.
dim_travel_package	39	Clean and complete. All packages have valid details including category, destination, and price.
fact_sales	1234	No missing values. Booking and travel dates are strings — should be converted to datetime.
dim_feedback	320	Full feedback entries, no missing scores. would_recommend is stored as string instead of boolean — check consistency.
dim_refund	30	Small subset of data — only 30 refunds. Clean and complete, but refund_percentage should be validated for outliers.

Null & Duplicate Checks

Sample Code:

```
# Checking for nulls

for name, df in {'Customers': customers, 'Sales': sales, 'Feedback': feedback, 'Refund': refund}.items():
    print(f"\n{name} missing values:")
    print(df.isnull().sum())

# Checking for duplicate booking_id or customer_id
print("Duplicate booking_id:", sales.duplicated('booking_id').sum())
print("Duplicate customer_id:", customers.duplicated('customer_id').sum())
```

Results:

1. Missing Values:

Table	Column(s) with Nulls	Count	Notes
Customers	email	43	~3.5% of customer records are missing an email. The missing data will be replaced with 'not provided' as agreed upon.
Others	<i>None</i>	0	All other key datasets (Sales, Feedback, Refund) are complete.

2. Duplicate Checks:

Table	Column Checked	Duplicates	Notes
Sales	booking_id	0	Each booking is uniquely recorded.
Customers	customer_id	0	Each customer is uniquely identified.

Key Distributions

Sample Code:

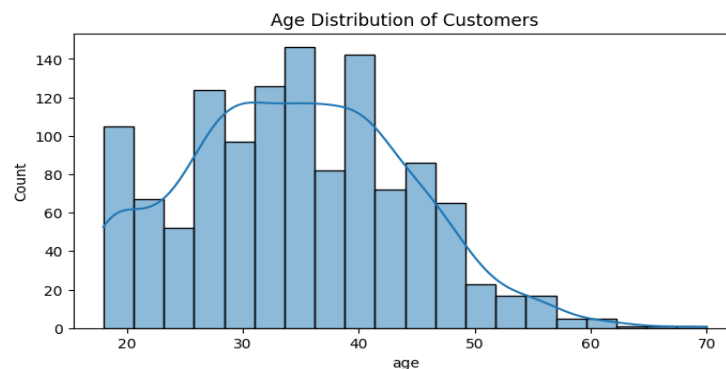
```
# Age distribution
plt.figure(figsize=(8,4))
sns.histplot(customers['age'], kde=True, bins=20)
plt.title("Age Distribution of Customers")
plt.show()

# Satisfaction score
sns.histplot(feedback['satisfaction_score'], bins=10, color='red')
plt.title("Satisfaction Score Distribution")
plt.show()

# Refund %
sns.histplot(refund['refund_percentage'], bins=10, color='skyblue')
plt.title("Refund Percentage Distribution")
plt.show()
```

Results:

1. Ages Distribution of Customers:



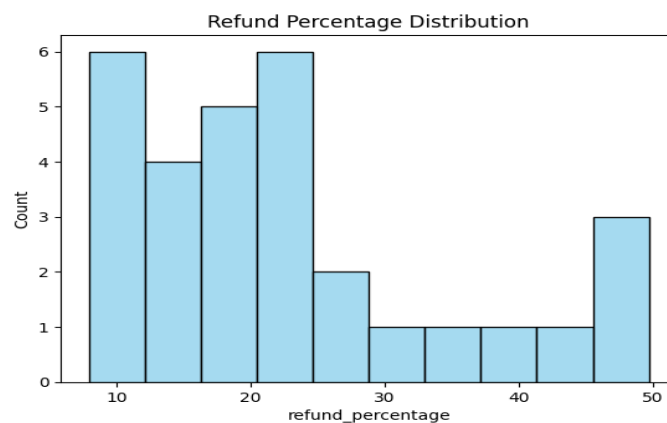
- The age of customers follows a right-skewed distribution, peaking around the 30–40 age group.
- There is a noticeable concentration between 25 and 45 years old, indicating the business primarily attracts young to middle-aged travellers.
- Very few customers are above 60, suggesting an opportunity to expand offerings to senior travellers if relevant.

2. Satisfaction Score Distribution:



- Most satisfaction scores are between 7 and 10, indicating high customer satisfaction overall.
- However, a few lower scores (1–4) are present, signaling a small group of dissatisfied customers.
- The score distribution leans positive, which is expected, but it's essential to analyze negative feedback to pinpoint areas of improvement (e.g., guides, itinerary, accommodation).

3. Refund Percentage Distribution:



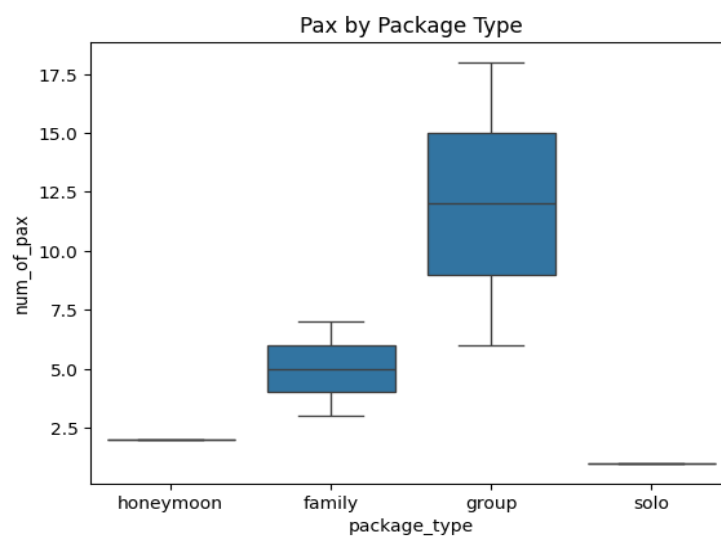
- Refund percentages vary but generally cluster around the lower end (10–30%).
- Higher refund percentages are less frequent but still occur, possibly tied to major issues or specific refund reasons (e.g., cancellations due to external factors).
- This pattern indicates that while refunds exist, most are partial refunds, which may help manage financial loss while maintaining customer goodwill.

Business Logic Checks

Sample Code:

```
# num_of_pax vs package_type
merged = sales.merge(travel_package[['package_id', 'package_type']], on='package_id')
sns.boxplot(x='package_type', y='num_of_pax', data=merged)
plt.title("Pax by Package Type")
plt.show()

# Check if any refunds have amount > total price
merged_refund = refund.merge(sales[['booking_id', 'package_id', 'num_of_pax']], on='booking_id')
merged_refund = merged_refund.merge(travel_package[['package_id', 'price_per_pax']], on='package_id')
merged_refund['expected_price'] = merged_refund['price_per_pax'] * merged_refund['num_of_pax']
merged_refund['is_outlier'] = merged_refund['refund_amount'] > merged_refund['expected_price']
print("Refund outliers (amount > total trip value):", merged_refund['is_outlier'].sum())
```



- The distribution of num_of_pax aligns well with the definitions of each package_type. No anomalies or illogical values (like solo bookings with more than 1 pax) are observed